

Project Report II – Cloud Computing

Cloud-Based Bug Tracking and Reporting System for Developers

Team number: 16

Student names: Wong Jun Yu Kean, Lee Wen Jie Brian, Lum Ko Sand,
Tan Si Han, Wee Hong Ru Curtis

1 April 2026

1. Introduction

The Cloud-Based Bug Tracking and Reporting System is a web application designed to help developers and users efficiently report, manage, and track software bugs in a centralized platform. Bug tracking systems play a critical role in software development by allowing teams to identify issues, monitor progress, and collaborate on solutions.

The goal of this project is to develop a fully functional cloud-based system that enables users to submit bug reports with detailed information such as title, description, severity level, and screenshots. The system also allows administrators to assign bugs to developers and update their status throughout the development lifecycle.

The application is deployed on a cloud platform using scalable and distributed infrastructure. It demonstrates key cloud computing concepts including load balancing, auto scaling, and secure data storage [1]. These features ensure that the system remains available, responsive, and secure even under varying workloads.

The scope of the project includes frontend development for user interaction, backend API implementation for system logic, and deployment using cloud infrastructure. Each team member contributed to different components such as frontend design, backend development, cloud deployment, database management, and system integration.

2. Architecture Design and Solutions

System Architecture

The system architecture consists of a client layer, application layer, and cloud infrastructure layer [1]. Users interact with the system through a web browser, and requests are routed through an Application Load Balancer [2]. The load balancer distributes traffic across multiple EC2 instances running the backend API. (Refer to Appendix I)

The backend instances process user requests and interact with Amazon RDS for structured data storage [4]. Although Amazon S3 was initially designed for storing uploaded screenshots, due to access and permission limitations, screenshots are instead stored within the EC2 instance storage. The corresponding screenshot file paths or links are then recorded in the RDS database for retrieval. CloudWatch is used to monitor system performance and resource utilization. (Refer to Appendix III, IV and VI)

This architecture ensures high availability, scalability, and reliability by distributing workloads and dynamically scaling resources. (Refer to Appendix I)

Functional Components

The system consists of the following key components:

1. Frontend Web Application
The frontend provides user interfaces for login, bug submission, and viewing bug reports. It communicates with the backend via API calls.
2. Backend API Server
The backend handles business logic such as processing bug submissions, updating bug status, and retrieving data from the database.
3. Cloud Database
A relational database is used to store bug reports, user accounts, and system data, ensuring persistence and reliability.
4. Cloud Storage
Screenshot data is stored using EC2 instance storage, while the corresponding file paths or links are maintained in the relational database. This approach was adopted due to limited access to object storage services.
5. Load Balancer
Distributes incoming traffic across multiple backend instances to improve availability and performance.
6. Auto Scaling Group
Automatically adjusts the number of backend instances based on system load, ensuring efficient resource usage [3].
7. Monitoring and Logging
Cloud monitoring tools are used to track system performance and detect potential issues.

Workflow Scenario

The system operates as follows:

1. A user logs into the system through the web interface
2. The user submits a bug report with details and optional screenshot
3. The request is sent to the backend API through the load balancer
4. The backend processes the request and stores the data in the database
5. If a screenshot is included, it is stored within the EC2 instance, and the file path or link is saved in the database
6. The bug report is displayed on the dashboard for tracking and updates

Deployment

The system is deployed on a cloud platform using virtual machines for backend services. Multiple instances are configured behind a load balancer to ensure high availability. (Refer to Appendix II)

An auto scaling group is implemented to dynamically adjust the number of instances based on traffic conditions [3]. The database is hosted separately to ensure data persistence. Due to access limitations to object storage services, file uploads such as screenshots are handled within EC2 instance storage, with references stored in the database.

3. Discussions

Scalability

The system achieves scalability through the use of an auto scaling group. When the number of incoming requests increases, additional backend instances are automatically launched to handle the load [3]. This ensures that the system can support multiple users simultaneously without performance degradation.

Reliability

Reliability is ensured through the use of a load balancer, which distributes traffic evenly across multiple backend instances [2]. In the event of a server failure, traffic is redirected to other available instances, ensuring continuous system availability.

Additionally, cloud monitoring tools are used to detect and respond to system issues, further improving reliability. (Refer to Appendix VI)

Security

Security is implemented through several mechanisms:

- User authentication to restrict access to authorized users
- Secure communication using HTTPS
- Database access control to prevent unauthorized access
- Isolation of resources within a cloud network

These measures help protect sensitive user data and ensure system integrity. (Refer to Appendix V)

Limitations and Future Improvements

Despite its functionality, the system has several limitations. The AI-based bug classification is relatively simple and can be improved using more advanced machine learning techniques.

One limitation of the current system is the use of EC2 instance storage for screenshot handling. Since instance storage may not be persistent across instance restarts or scaling events, this may lead to potential data loss or inconsistency.

In addition, the system can be enhanced by implementing real-time notifications, improved user interfaces, and more advanced analytics for bug tracking.

Future work may also include integrating Amazon S3 for object storage, continuous integration tools and expanding system scalability for large-scale deployment.

4. Reflection and takeaways

Throughout this project, the team gained valuable experience in designing and deploying a cloud-based application [6]. The project provided practical exposure to cloud computing concepts such as load balancing, auto scaling, and distributed system design.

The team also improved their skills in full-stack development, including frontend design, backend API development, and database management. Collaboration among team members was essential in integrating different components into a fully functional system.

One key takeaway is the importance of proper system architecture in ensuring scalability and reliability. The project also highlighted the challenges of integrating multiple cloud services and ensuring secure communication between components.

Overall, the project successfully demonstrated the use of cloud technologies to build a functional and scalable web application.

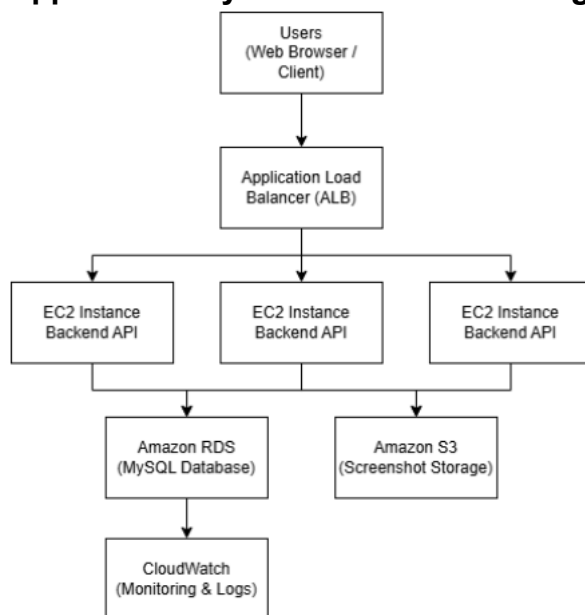
5. Contribution from team members

SN	Name of Team Member	Student ID	Responsible Components	Contribution (%)
1	Wong Jun Yu Kean	2301234	Frontend development (User interface, bug submission forms, dashboard)	20%
2	Lee Wen Jie Brian	2301261	Backend API development and system logic	20%
3	Lum Ko Sand	2301263	AI or automated bug classification and feature integration	20%
4	Tan Si Han	2301264	Cloud infrastructure deployment, load balancing, and auto scaling	20%
5	Wee Hong Ru Curtis	2301266	Database design, cloud storage integration, and security configuration	20%
Total:				100%

Reference

- [1] Amazon Web Services, "What is Cloud Computing?" [Online]. Available: <https://aws.amazon.com/what-is-cloud-computing/>
- [2] Amazon Web Services, "Elastic Load Balancing," [Online]. Available: <https://aws.amazon.com/elasticloadbalancing/>
- [3] Amazon Web Services, "Amazon EC2 Auto Scaling," [Online]. Available: <https://aws.amazon.com/autoscaling/>
- [4] Amazon Web Services, "Amazon RDS for Relational Databases," [Online]. Available: <https://aws.amazon.com/rds/>
- [5] Amazon Web Services, "Amazon S3 – Object Storage," [Online]. Available: <https://aws.amazon.com/s3/>
- [6] National Institute of Standards and Technology (NIST), "The NIST Definition of Cloud Computing," 2011.
- [7] SIT CSD3156 Cloud Computing Lecture Notes, Singapore Institute of Technology, 2026.

Appendix I - System Architecture Diagram



Appendix II - EC2 Instances

The screenshot displays the AWS Management Console interface for EC2 instances. At the top, there's a navigation bar with the AWS logo, account information (United States (N. Virginia)), and user details (voclab/user4017251=WEE_HONG_RU_CURT). Below this, the left-hand navigation pane shows various services like Dashboard, AWS Global View, Events, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, and Capacity Manager. The main area is titled "Instances (1/11)" and includes a search bar and filter options. A table lists three instances:

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IP
☐		i-00161b17521d2e0bc	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	-	-
☐		i-0e0aa5af1c491aa3	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	-
☐	WebServer2	i-0d54cea0070dfc1db	Stopped	t2.micro	-	View alarms +	us-east-1b	-	-
☒	WebServer1	i-0b5ca6c8fbbaa702e	Stopped	t2.micro	-	View alarms +	us-east-1a	-	-

Below the table, the selected instance "i-0b5ca6c8fbbaa702e (WebServer1)" is detailed. It shows tabs for Details, Status and alarms, Monitoring, Security, Networking, Storage, and Tags. Under the "Details" tab, the "Instance summary" section provides key information:

- Instance ID:** i-0b5ca6c8fbbaa702e
- IPv6 address:** -
- Hostname type:** IP name: ip-10-0-11-141.ec2.internal
- Answer private resource DNS name:** -
- Public IPv4 address:** -
- Instance state:** Stopped
- Private IP DNS name (IPv4 only):** ip-10-0-11-141.ec2.internal
- Instance type:** t2.micro
- Private IPv4 addresses:** 10.0.11.141
- Public DNS:** -
- Elastic IP addresses:** -

Appendix III - RDS Dashboard

The screenshot displays the AWS Management Console interface for configuring an Amazon RDS database instance. The top navigation bar shows the user's location as United States (N. Virginia) and their account ID. The left-hand menu includes options like Dashboard, Databases, Query editor, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, Event subscriptions, and Recommendations.

The main content area is titled "Aurora and RDS" and "Databases". It shows the configuration for a database instance named "database-1". Key settings include:

- Internet access gateway:** Disabled
- IAM Authentication:** Disabled
- Programming language:** MySQL (macOS)
- Endpoint type:** Instance endpoint

The "Connection steps" section provides instructions and commands for connecting to the database:

- curl -I -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
- mysql -h database-1.cjkq6d2pxv.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem

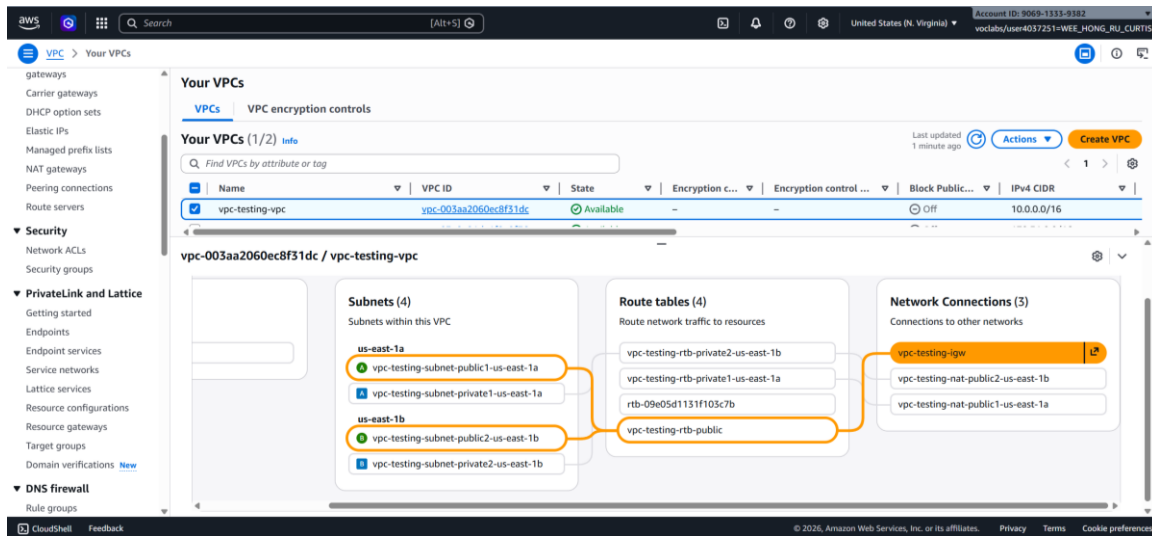
The "Connected compute resources" section indicates that connections to compute resources created automatically by RDS are shown, while manually created ones are not. A filter box allows searching by compute resource name.

Resource identifier	Resource type	Availability Zone	VPC security group	Compute resource security group	Connected proxy
i-00161b17521d2e0bc	EC2 Instance	us-east-1b	rds-ec2-2	ec2-rds-2	-
i-0b5ca6c8fbaa702e	EC2 Instance	us-east-1a	rds-ec2-2	-	-
i-0d54cea0070dfcf1db	EC2 Instance	us-east-1b	rds-ec2-2	ec2-rds-2	-
i-0e0a8aa5f1c491aa3	EC2 Instance	us-east-1a	rds-ec2-2	ec2-rds-2	-

Appendix IV - Screenshot Storage in EC2

[illegible]

Appendix V - VPC Dashboard

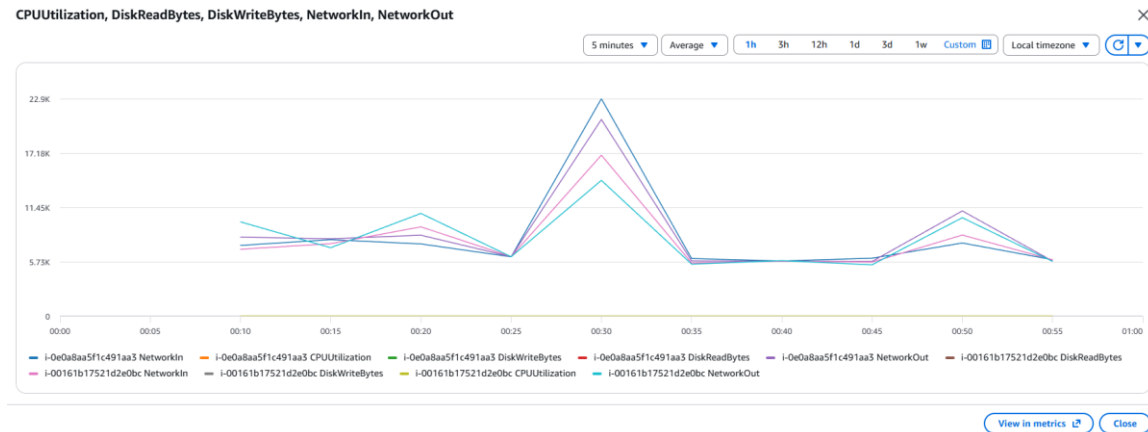


The screenshot shows the AWS VPC Dashboard for the 'vpc-testing-vpc' (VPC ID: vpc-003aa2060ec8f31dc). The dashboard is organized into several sections:

- Your VPCs:** A table listing the VPC with columns for Name, VPC ID, State, Encryption controls, Block Public access, and IPv4 CIDR. The VPC is in an 'Available' state with a CIDR of 10.0.0.0/16.
- Subnets (4):** A list of subnets within the VPC, categorized by availability zone (us-east-1a and us-east-1b). Each subnet has a status icon (green for available, blue for in-use).
- Route tables (4):** A list of route tables associated with the VPC, including private and public route tables.
- Network Connections (3):** A list of network connections, including an Internet Gateway (igw) and two NAT gateways.

The dashboard also includes a left-hand navigation menu with options like Gateways, Security, PrivateLink and Lattice, and DNS Firewall. The top bar shows the account ID and region (United States (N. Virginia)).

Appendix VI - CloudWatch Dashboard



END OF REPORT